

PyIDFix: Bridging the Gap in Implementation-Dependent Flaky Test Detection and Repair for Python

Brandon Li, Jenny Chung
University of Illinois at Urbana-Champaign
[bcli3,jennyc3]@illinois.edu

Abstract

Flaky tests pass or fail non-deterministically across CI runs without changes to the code. These types of tests undermine software development by eroding developer trust, slowing CI/CD pipelines, and masking regressions that reach production. One prominent yet elusive category of flaky tests is implementation-dependent (ID) tests, which fail due to incorrect assumptions about under-constrained APIs (e.g., hash map iteration order) or environments (e.g., platform quirks or timing subtleties).

Much prior work focuses on Java projects, with tools emphasizing broad flakiness or order-dependent (OD) tests via test reordering. Although Python-specific flaky test studies exist, they primarily target OD flakiness. ID tests remain prevalent in Python’s dynamic standard library and third-party APIs, yet they are largely undetected and unaddressed because they require deep code introspection.

To bridge this gap, we present PyIDFix for automatically detecting and fixing Python ID tests. By statically analyzing code internals and systematically varying execution environments to expose nondeterministic API behavior, our approach identifies wrong implementation assumptions and generates targeted deterministic fixes to restore test reliability.

1 Artifact Information

Link to the GitHub Repository: <https://github.com/jenny-chung/CS527-Research-Project>

2 Introduction

To detect bugs and regressions in software development, developers run automated tests that execute the software under test (SUT) and verify results against expected outcomes. Passing tests provide confidence when merging changes, while failing tests prompt debugging of recent modifications to identify and resolve issues. Reliable testing therefore underpins robust software development by verifying correctness amid rapid changes and enabling confident CI/CD iteration. However, flaky tests, which are tests that nondeterministically pass or fail on the same code version, can weaken this assurance. By undermining the core premise of regression testing, where failures signal real problems, flaky tests may mislead developers into chasing false alarms rather than genuine regressions.

Flaky tests are widely recognized as a major challenge in software development. They are not only problematic, but also prevalent in large codebases, where even a small percentage of flakiness can translate into substantial productivity losses. For example, a study from Google [1] found that flaky tests account for 4.56% of test failures and consume over 2% of coding time. Similar industry observations report that flaky tests increase CI reruns, debugging

overhead, and reduce developer trust in automation. These hidden costs compound across the software development lifecycle. Despite growing awareness of flaky tests, certain categories such as ID tests in Python, remain underexplored, motivating the need for targeted detection and repair techniques.

Extensive research has targeted the detection and mitigation of flaky tests, recognizing them as a major challenge to test reliability and developer productivity. Much of this work has focused on Java, where large datasets have enabled the development of sophisticated techniques for identifying and repairing flaky behavior. Approaches leverage strategies such as repeated test execution, dependency analysis, and runtime monitoring to expose nondeterministic outcomes. For instance, iDFlakies focuses on detecting and classifying flaky tests using test reordering and reruns [2], while Nondex dynamically analyzes test executions in Java to detect implementation-dependent flakiness [3]. More recent tools, such as DexFix, extend these approaches to automatically repair wrong assumptions on underdetermined specifications, specifically targeting ID flakiness in Java tests [4]. Together, these advances have significantly improved the understanding of flaky tests and established a strong foundation for addressing flakiness in the Java world.

In contrast, research on flaky tests in Python remains limited. One large-scale study [5] found that flakiness is as prevalent in Python as in Java, with order dependency accounting for 59% of flaky tests and 28% stemming from test infrastructure issues. The remaining failures were largely associated with network and randomness APIs, highlighting the role of nondeterministic behavior in Python. These findings have guided much of the existing Python literature toward order-dependent flakiness, emphasizing detection through randomized test ordering or rerun strategies, with comparatively less focus on automated repair. While existing work provides valuable insight into Python flaky behavior, they explore a narrower portion of the problem space compared to Java. Notably, ID flaky tests in Python remain largely underexplored, leaving a gap in techniques for both detecting and automatically repairing this class of failures.

Although existing work has advanced the understanding of flaky tests, a critical gap remains in addressing ID flaky tests in Python. Most techniques, including Java-centric approaches such as DexFix, cannot be ported to Python due to differences in runtime behavior and test infrastructure [4]. Research on Python flaky tests has primarily emphasized OD flakiness, leaving failures arising from incorrect assumptions on underdetermined APIs largely underexplored. Existing Python tools mainly prioritize detection over repair, and repair tools such as iPFlakies are limited to fixing OD flakiness through test reordering [6]. Large language model (LLM) efforts like NIODebugger target non-OD flakiness, but lack Python ID

specialization [7]. These limitations cause existing techniques to miss subtle sources of nondeterminism inherent in Python’s runtime, leaving developers with little support for addressing ID flaky tests. Since no technique to date provides specialized detection and automated repair for this class of failures, this gap calls for techniques that can reliably detect and automatically repair ID tests while preserving their original intent.

The relevance of this problem extends to modern Python projects. While Python 3.7+ guarantees dictionary insertion order, set iteration remains non-deterministic, and unseeded randomness and floating-point equality are unaffected by any Python version. Moreover, a 2026 empirical study of flaky tests in quantum software repositories, including Qiskit, explicitly identifies unordered collection iteration as an active flakiness category and notes that even where a Python upgrade would resolve dictionary ordering, dependency constraints prevent many real-world projects from doing so in practice [13]. This confirms that ID flakiness in Python remains an open and actively researched problem.

To bridge this research gap, we present PyIDFix for automatically detecting and fixing ID tests in Python. Repairing ID flakiness is inherently challenging because it requires reasoning about code internals and identifying subtle assumptions about runtime behavior that are often difficult to detect through traditional testing approaches. Our approach consists of four key components: a static pattern analyzer that identifies suspicious patterns in test code such as unordered collection iteration, unseeded randomness, or floating-point equality assertions; an execution variant generator that runs tests under controlled environmental variations to confirm nondeterministic behavior; a deterministic patch generator that produces rule-based fixes, such as sorting unordered iterations or seeding random operations, while preserving test intent; and a determinism validator that verifies patched tests against the same environmental variations to ensure consistent outcomes. Implemented as a lightweight pytest plugin, the technique integrates into existing Python CI workflows and supports the repair of flaky ID tests without altering their semantic intent.

Our experimental methodology consists of three main stages: dataset construction, tool implementation, and systematic testing. In the dataset construction stage, we used an LLM to generate 5-10 self-contained Python programs. For each program, we created a pytest test suite containing both deterministic tests and intentionally flaky tests with specific known patterns (dictionary/set iteration without sorting, random calls without seeds, and exact float comparisons). Each flaky test was documented with its root cause and expected fix to serve as ground truth. In the implementation stage, we developed the tool’s four core components: a Static Pattern Analyzer using Python’s AST module to detect suspicious code patterns, an Execution Variant Generator that runs tests under different PYTHONHASHSEED values, a Deterministic Patch Generator that applies rule-based transformations (adding sorted(), injecting seeds, using pytest.approx(), etc.), and a Determinism Validator that re-executes patched tests to confirm fix effectiveness. In the testing stage, we validated the tool on our controlled dataset to establish baseline performance. All experiments were conducted on Ubuntu 24.04 with Python 3.12, and we systematically varied

PYTHONHASHSEED across 10 different values (0-9) for variant execution. We logged all detection results, variant execution outcomes, generated patches, and validation results.

Our experiments across 6 benchmark programs and 14 labeled flaky tests demonstrate that PyIDFix can effectively detect, repair, and validate implementation-dependent flaky tests. The Static Pattern Analyzer detected 13 of 14 tests (93%), outperforming a naive rerun baseline by 36 percentage points overall and by a full 100 percentage points on float equality tests—a pattern class that runtime methods cannot detect at all because float arithmetic is deterministic on a single machine. The Execution Variant Generator confirmed flakiness through mixed-outcome detection at a rate comparable to naive reruns for hash-sensitive patterns (71% for unordered iteration), while offering the added benefit of full reproducibility across environments. Of the 13 detected tests, the Deterministic Patch Generator successfully repaired all 13 (100% repair rate), applying targeted transformations—sorted() wrapping, random.seed() insertion, and pytest.approx() substitution—that raised the average variant-seed pass rate from as low as 13% to 100% across all pattern categories. The Determinism Validator further confirmed that all 10 correctly written reference tests remained stable after patching, showing that the repairs are narrowly scoped and do not introduce regressions. The one unrepaired test (test_random_values) was also undetected, as its randomness originates in an external helper function that the static analyzer does not trace into, highlighting the tool’s current limitation to direct, intra-file pattern detection.

List of Contributions:

- **A static analyzer for implementation-dependent flaky tests.** We present a pattern-matching analyzer built on Python’s abstract syntax tree (AST) that identifies three families of implementation-dependent flakiness: unordered hash-based collection iteration, unseeded pseudo-random number generation, and imprecise floating-point equality.
- **A variant-based execution engine for flakiness confirmation.** We design an execution variant generator that drives pytest in isolated subprocesses under systematically varied PYTHONHASHSEED environments, providing empirical evidence of non-deterministic behavior to complement static findings.
- **A rule-based patch generator for automated repair.** We implement a patch generator that applies targeted source-level transformations to eliminate detected flakiness: wrapping unordered collection accesses with sorted(), inserting random.seed() calls, and replacing exact floating-point comparisons with pytest.approx().
- **A determinism validator for patch correctness.** We provide a validator that re-executes patched tests under the same environmental variants used during detection, confirming that applied fixes both eliminate non-determinism and preserve test correctness.
- **A reproducible labeled benchmark suite.** We construct a benchmark of synthetic programs spanning all three flakiness pattern families, each annotated with a

ground_truth.json file specifying the flaky test identifiers, pattern types, root causes, and expected fixes, enabling automated and reproducible evaluation.

- **An end-to-end developer-facing pipeline.** We deliver a command-line interface (`python -m pyidfix demo`) that narrates the full detect→confirm→patch→validate pipeline with a unified diff, per-seed pass/fail grid, and machine-readable JSON output, making the tool’s behavior transparent and auditable.

3 Technique

3.1 Approach Overview

Our tool consists of four main components that work together to identify and repair implementation-dependent flaky tests in Python. The high-level architecture and the interaction between these stages are illustrated in Figure 1. Each component is designed to address a specific challenge in the detection-to-repair pipeline, ranging from environment manipulation to code-level patching.

The first component is the Static Pattern Analyzer. This component performs static code analysis on test files to identify suspicious patterns that may lead to nondeterministic behavior. Unlike tools that focus on test execution order, our analyzer targets inherent implementation-dependent behaviors within individual tests. The analyzer’s detection patterns include unordered collection iteration, hash-dependent operations, unseeded randomness, and floating-point equality assertions.

The second component is the Execution Variant Generator. Rather than repeatedly running the same test multiple times, which can miss implementation-dependent flakiness, this component executes tests under controlled environmental variations to expose nondeterministic behavior. Variant strategies include hash seed variation across seeds 0-9. Each suspicious pattern identified by the Static Pattern Analyzer is paired with appropriate execution variants. The component tracks which variants produce different outcomes, confirming the test’s flakiness.

The third component is the Deterministic Patch Generator. This component generates rule-based fixes for confirmed flaky tests, focusing on making execution deterministic while preserving test intent. For instance, for a flaky test using unordered iteration, the tool wraps the iteration in a `sorted()` call with appropriate key functions. Any sets are converted to sorted lists before iteration, and dictionary iteration is replaced with `sorted(dict.items())`.

The fourth component is the Determinism Validator. This component verifies that applied patches successfully eliminate flakiness by re-executing tests under the same environmental variations used during detection. The process re-runs the patched tests with multiple hash seeds, confirms consistent pass/fail outcomes, and flags tests that still exhibit nondeterministic behavior for manual review.

3.2 Approach Details

Component 1: Static Pattern Analyzer

The Static Pattern Analyzer is the entry point of the PyIDFix pipeline. It examines Python test source code without executing it, using the standard library’s `ast` module to parse the source into an Abstract Syntax Tree (AST) and then walk that tree searching for

three families of implementation-dependent flakiness patterns.

Algorithm 1: Stateful Detection of Unseeded Randomness

Input: AST FunctionDef node F

Output: Set of findings R

```

1  $P \leftarrow \emptyset$ ; // Pending random calls
2  $SeedFound \leftarrow false$ ;
3 foreach node  $n$  in  $F$  via GENERICVISIT do
4   if  $n$  is a random.seed call then
5      $SeedFound \leftarrow true$ ;
6   else if  $n$  is any random.* call then
7      $P \leftarrow P \cup \{n\}$ ;
8 if  $SeedFound = false$  then
9   foreach call  $c \in P$  do
10     $\text{emit FINDING}(\text{unseeded\_randomness}, c.\text{lineno})$ ;
```

The core data structure is the Finding dataclass, which records a pattern name (`unordered_iteration`, `unseeded_randomness`, or `float_equality`), the line and column number of the offending code, a human-readable message, a short code snippet, and the name of the enclosing test function. Findings are returned as a list by `analyze_file(path)` or `analyze_source(source)`, giving the rest of the pipeline a structured, per-function view of what needs to be fixed.

Detection is performed by `PatternVisitor`, a subclass of `last.NodeVisitor`. The visitor maintains three pieces of state across the traversal: `_current_function` (the name of the function currently being visited), `_env` (a dictionary mapping local variable names to their inferred types, e.g., "set" or "dict"), and `_random_calls_pending` (a list of `random.*` call nodes collected within the current function). A `_seen` set prevents the same finding from being reported twice.

Unordered iteration detection operates across three AST node types. In `visit_Assign`, when the right-hand side is a `list()` call, the analyzer inspects the argument. If the argument is a direct dict method call (`.items()`, `.keys()`, `.values()`), a set literal, a previously tracked set or dict variable (looked up in `_env`), or a function call whose name contains keywords like "key", "value", "item", "set", or "dict", a finding is emitted. In `visit_For`, if the iterable of the loop is a method call on a dict without an enclosing `sorted()`, the loop is flagged. In `visit_Call`, if the call is `iter(s)` or `next(iter(s))` where `s` is a variable tracked as a set in `_env`, an unordered iteration finding is emitted. The `_env` is populated in `visit_Assign` whenever a variable is bound to a set literal, a dict literal, or a dict comprehension over a set variable, allowing the analyzer to track collection types across statement boundaries within a function.

Unseeded randomness detection uses a two-pass approach within each function scope, as shown in Algorithm 1. When `visit_FunctionDef` is entered, `_random_calls_pending` and `_random_seed_in_func` are reset. As `visit_Call` encounters attribute calls of the form `random.`, they are accumulated in `_random_calls_pending`. If a call matching `random.seed(...)` is encountered anywhere in the function, `_random_seed_in_func` is set to `True`. When the function’s traversal completes, if no seed was seen, all accumulated

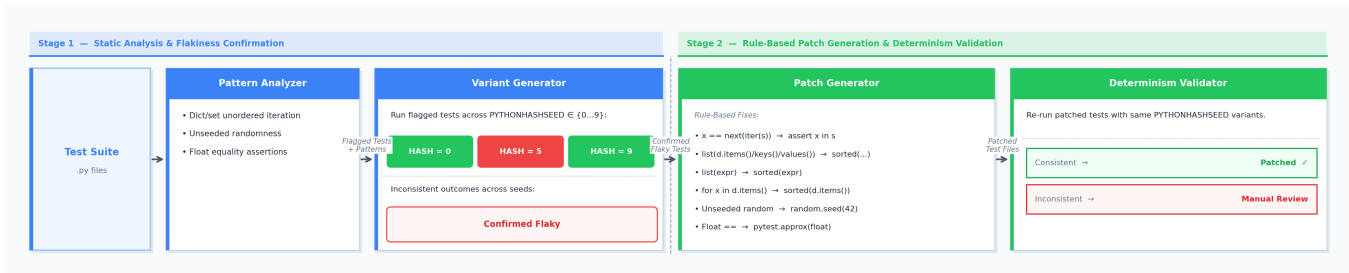


Figure 1: Overview of the PyIDFix detection and repair pipeline. The architecture illustrates the flow from initial pattern identification and hash-seed-based confirmation to automated patching and final consistency verification

`random.` calls are emitted as `unseeded_randomness` findings. This approach correctly handles cases where `random.seed()` is called before use (safe) but not cases where no seed is present at all.

Float equality detection operates in `visit_Assert`. If the assert’s test is a comparison of the form `x == <float literal>`, where the right-hand comparator is a floating-point constant node, a `float_equality` finding is emitted at the assert’s line.

Component 2: Execution Variant Generator The Execution Variant Generator provides empirical confirmation that a flagged test actually behaves non-deterministically across environments. Rather than modifying the Python runtime or injecting mocks, it drives the existing pytest infrastructure by spawning isolated subprocesses with different environment variables, observing whether outcomes differ.

The primary function, `run_test_with_variants(test_path, hash_seeds, ...)`, accepts a pytest-style node ID (i.e. `tests/test_flaky.py::test_foo`) and a list of integer hash seeds. For each seed, it constructs a copy of the current OS environment, sets `PYTHONHASHSEED` to that seed value, and calls `_run_pytest`, which invokes `python -m pytest -v -tb=short -q` as a subprocess using Python’s `subprocess.run` with a 60-second timeout. Each run produces a `VariantResult` dataclass containing the seed label, a boolean `passed` (determined by `returncode == 0`), and the captured `stdout` and `stderr`.

`PYTHONHASHSEED` is the key environmental lever for this tool. CPython randomizes the hash values of strings and bytes objects at interpreter startup using this seed. As a result, the iteration order of sets and dictionaries (whose internal buckets depend on hash values) varies between runs with different seeds. By sweeping seeds 0 through $N - 1$ and observing whether any run fails while others pass, the generator detects tests that make order-sensitive assumptions about hash-based collections.

The `is_flaky(results)` function examines the list of `VariantResult` objects and returns `True` if the set of passed booleans contains both `True` and `False` and if the test’s outcome is not uniform across all variants. A consistent all-fail result (the test always fails) is not classified as flakiness by this check; it indicates a pre-existing test failure rather than environment-sensitive non-determinism. Figure 2 illustrates a representative variant execution output, where 8 of 10 seeds produce a failure while 2 pass, confirming implementation-dependent flakiness.

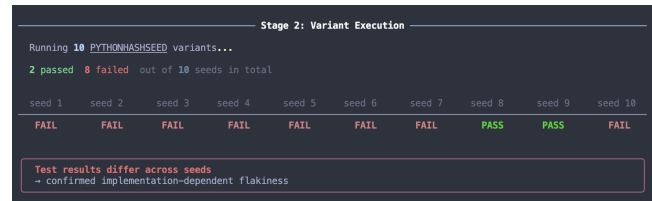


Figure 2: Confirmation of implementation-dependent flakiness via variant execution. The divergence between passing (2) and failing (8) results across different hash seeds empirically validates the non-deterministic nature of the test logic

Working directory resolution is handled by `_infer_cwd`, which detects whether the test path is inside an `eval_dataset/programs//` directory and, if so, sets the working directory to the program root rather than the repository root. This is necessary because test files import from local source `.py` modules, and pytest must be launched from a directory where those imports resolve correctly. `_pytest_target_for_cwd` then converts the absolute test path to a path relative to the chosen working directory for use in the subprocess command.

Component 3: Deterministic Patch Generator

The Deterministic Patch Generator transforms the source code of a flaky test into a deterministic equivalent by applying targeted, rule-based text edits guided by the findings produced by the Static Pattern Analyzer. It operates purely on source text and returns a modified source string; it does not re-parse the result or validate that the patched code passes.

The top-level function is `generate_patch(source, findings)`. To avoid line-number invalidation caused by inserting or removing lines, it sorts findings in descending line order (bottom-up) before applying them. Each finding is dispatched to one of three pattern-specific patch functions.

Patching unordered iteration (`_patch_unordered_iteration`) isolates the flagged line and applies a prioritized series of text substitutions. First, if the line contains `next(iter(s))`, a regex match replaces it with a membership test: `assert x in s`, which preserves the original intent of verifying that a specific element is present in the collection without imposing an arbitrary sort order. Second, if the line contains `list(x.items())`, `list(x.keys())`, or `list(x.values())`, a regex replaces `list(...)` with `sorted(...)`

```

1 def test_set_iteration():
2     s = {"apple", "banana", "cherry"}
3     result = list(s) # Flaky: order varies by hash
4     seed
5     assert result == ["apple", "banana", "cherry"]

```

Figure 3: Original flaky test (unordered iteration).

```

1 def test_set_iteration():
2     s = {"apple", "banana", "cherry"}
3     result = sorted(s) # PyIDFix patch
4     assert result == ["apple", "banana", "cherry"]

```

Figure 4: Patched deterministic test (sorted iteration).

while preserving the method call. Third, as demonstrated in the transition from Figure 3 to Figure 4, if the line contains a generic `list(...)` call that was flagged (e.g., `list(some_set_var)`), the `list(` prefix is replaced with `sorted(` directly. Finally, if the flagged line is a for loop iterating over a dict method call, a regex wraps the iterable with `sorted(...)`.

```

1 def test_random_shuffle():
2     import random
3     lst = [1, 2, 3]
4     random.shuffle(lst)
5     assert lst == [2, 1, 3] # Flaky: shuffle
6     depends on global state

```

Figure 5: Original flaky test (unseeded randomness).

```

1 def test_random_shuffle():
2     import random
3     random.seed(42) # PyIDFix patch
4     lst = [1, 2, 3]
5     random.shuffle(lst)
6     assert lst == [2, 1, 3] # Deterministic:
7     matches seed 42

```

Figure 6: Patched deterministic test (seeded randomness).

Patching unseeded randomness (`_patch_unseeded_randomness`) re-parses the source with `ast.parse` to locate the `FunctionDef` node whose name matches the finding’s `function_name`. It identifies the first statement in the function body (skipping the docstring) and determines its indentation from the source text. The function then scans the next five lines of the function body for an existing `import random` or `random.seed` statement. As illustrated in the transition from the flaky state in Figure 5 to the repaired state in Figure 6, the tool performs a strategic insertion. If `import random` is found and no `random.seed` follows it, a `random.seed(42)` line is inserted immediately after the `import`. If neither is found, `random.seed(42)` is inserted at the top of the function body. This insertion makes

the pseudo-random number generator’s state deterministic and reproducible from that point forward in the test.

Patching float equality (`_patch_float_equality`) applies a regex to the flagged assert line, matching the pattern `assert ==`. It replaces the right-hand side with `pytest.approx()`, which performs a comparison with a relative tolerance of $1e-6$ by default. If `import pytest` is not already present in the source, it is inserted before the first non-comment, non-blank line of the file.

Ultimately, PyIDFix presents these fixes to the user via a command-line interface. As shown in Figure 7, the tool generates a unified diff that clearly highlights the transition from non-deterministic code (e.g., unordered set iteration) to a stable, deterministic equivalent (e.g., sorted iteration), ensuring the test remains reliable across different execution environments.

Figure 7: The PyIDFix CLI generates a unified diff to repair ID-flakiness by automatically replacing non-deterministic set iteration with a deterministic `sorted()` transformation

Component 4: Determinism Validator The Determinism Validator is the final stage of the pipeline. Its role is to confirm that the patch produced by the Patch Generator has successfully eliminated flakiness — that is, the patched test now produces a consistent passing outcome across all environmental variants that previously triggered non-determinism.

The validator’s interface is `validate_patched_test(test_path, hash_seeds, cwd)`. It delegates directly to `run_test_with_variants` from the Execution Variant Generator, using the same set of hash seeds (default 0–9) to run the patched test file under the same range of `PYTHONHASHSEED` values. It then applies two acceptance criteria: all variant runs must have passed = `True` (`all_passed`), and `is_flaky` must return `False` (outcomes must be uniform). Both conditions must hold for the validator to return `True`. If the test consistently fails across all seeds, indicating that the patch broke the test logic rather than fixing non-determinism, the validator returns `False` and the failure is distinguished from residual flakiness in the pipeline output.

The validator intentionally reuses the exact same subprocess execution infrastructure as the pre-patch variant generation. This ensures the comparison is apples-to-apples: the same seeds, the same working directory resolution, and the same `pytest` invocation are used before and after patching. Any change in behavior between the two sweeps can be attributed specifically to the patch rather than to environmental differences in test execution.

A secondary function, `validate_patched_file(file_path, test_ids, hash_seeds, cwd)`, allows batch validation of multiple tests from a single patched file. It iterates over a list of test function names, constructing a full `pytest` node ID for each, calling `validate_patched_test` for each one individually, and returning

a dictionary mapping each test ID to its (passed, results) outcome. This is used internally by the evaluation script when multiple tests within the same file are being assessed.

4 Experimental Evaluation

This work aims to answer the following research questions (RQs):

- How effectively can static analysis identify implementation-dependent flaky tests compared to traditional rerun-based approaches?
- How does variant-based execution compare to repeated reruns in confirming test flakiness?
- What percentage of detected flaky tests can be automatically repaired using rule-based transformations?
- Do the generated patches preserve test correctness while eliminating flakiness?

4.1 Evaluation Plan

To assess the effectiveness of our tool, we conducted an evaluation using a controlled benchmark dataset. We generated a ground-truth dataset of 6 Python programs with intentionally injected flaky tests covering various flakiness patterns: unordered dictionary/set iteration, unseeded randomness, and floating-point equality issues. Each program included a mix of deterministic baseline tests and flaky tests with known patterns and documented expected fixes. This controlled setting allowed us to precisely measure detection recall across pattern categories, confirmation effectiveness through variant execution, repair success rates for each pattern type, and validate that patches preserve test semantics. We compared our variant-based confirmation approach against a naive rerun-based baseline (running each test ten times with a randomized PYTHONHASHSEED) to evaluate efficiency gains.

Due to time constraints, our evaluation was limited to a synthetically generated benchmark. Applying PyIDFix to real-world repositories, including scientific computing libraries such as SciPy, NumPy, and pandas or developer tools like pytest itself, would test generalizability across complex inter-module dependencies and a broader diversity of ID flakiness patterns, and remains an important direction for future work.

4.2 Evaluation Results

RQ1: Effectiveness of Static Analysis for Flaky Test Detection

Table 1 summarizes the detection rate of PyIDFix’s Static Pattern Analyzer against a naive rerun-based baseline across the three flakiness patterns in our benchmark. The naive baseline runs each test ten times with a randomized PYTHONHASHSEED and reports a test as flaky only when it observes mixed pass/fail outcomes. Figure 8 visualizes these rates alongside the per-program analysis latency.

The Static Pattern Analyzer detects 13 of 14 labeled tests (93%), compared to 57% for the naive rerun baseline, a 36 percentage point advantage overall. The single missed detection is `test_random_values` (`prog_02`), whose `random.randint` call resides inside the external helper `get_random_values()` in `source.py`. Because the analyzer operates only on the test file and does not perform interprocedural tracing, this indirect usage falls outside its scope. This suggests that the tool’s recall could be improved by

Table 1: Detection rate by method and pattern (14 labeled flaky tests).

Pattern	n	Static	Naive Reruns	Advantage
Unordered iteration	7	100%	71%	+29 pp
Unseeded randomness	4	75%	75%	0 pp
Float equality	3	100%	0%	+100 pp
Overall	14	93%	57%	+36 pp

incorporating lightweight call-graph analysis to follow function calls from the test into helper modules.

The most striking observation is the 100-percentage-point gap on *float equality* tests, where static analysis achieves 100% detection and the naive rerun baseline achieves 0%. The root cause lies in the deterministic nature of floating-point arithmetic on a single machine: a test comparing a computed float to a literal either always passes or always fails across reruns, so the baseline never observes the mixed outcomes it requires to flag a test as flaky. Static analysis is immune to this limitation because it examines the *structure* of the assertion (an exact `==` comparison on a float expression) rather than the runtime outcome. This highlights a fundamental blind spot of rerun-based approaches: they cannot detect latent correctness issues that happen to be consistent on the local platform but would manifest on a different architecture or Python version.

For *unordered iteration*, the static analyzer achieves 100% while naive reruns reach only 71%. The two missed tests for the baseline both fail on *all* ten random seeds, meaning the iteration order they depend on happens never to match the expected value across the sampled seeds. To the baseline this looks like a consistently failing test rather than a flaky one, so it goes unreported. Critically, there is no way for the baseline to distinguish this case from a legitimately broken test without examining the source.

Analysis latency is sub-millisecond for every program (see Figure 8), confirming that static analysis adds negligible overhead relative to even a single test execution. This cost gap would widen substantially on larger test suites, where a rerun-based baseline must execute each test many times while the static analyzer scans the source files in microseconds.

RQ2: Variant-Based Execution vs. Naive Reruns

Figure 9 compares the flaky-test detection rate of PyIDFix’s Execution Variant Generator, which sweeps ten deterministic PYTHONHASHSEED values (0–9), against the same naive rerun baseline used in RQ1. A test counts as detected by a runtime method only when it exhibits *mixed outcomes* (at least one passing and one failing run) across the ten executions.

For *unordered iteration*, both methods detect 71% of tests, with each method missing a different subset. Variant sweep misses two tests that fail on *all* of seeds 0–9, while naive reruns miss two tests that fail on all ten random seeds. This divergence suggests that the set of seeds 0–9 does not uniformly cover the space of possible hash orderings. Certain iteration orders may only appear under seeds outside that small window. Expanding the sweep to a wider or randomly sampled set of seeds would likely improve coverage, though at the cost of longer run times.

For *unseeded randomness*, naive reruns outperform the variant sweep (75% vs. 50%). The root cause is a mismatch between the tool’s mechanism and the pattern: `random.choice`, `random.sample`, and `random.shuffle` draw from Python’s `random` module state, which is seeded independently of `PYTHONHASHSEED`. Varying the hash seed therefore has no effect on the outcome of these tests, so the variant sweep cannot expose their flakiness. Naive reruns happen to call into different `random` module states across executions, which occasionally produces the mixed outcomes needed for detection. This is an inherent limitation of using `PYTHONHASHSEED` as the sole variation axis; a more complete tool would additionally vary `random.seed()` across runs.

For *float equality*, both methods score 0%, reinforcing the conclusion from RQ1 that no runtime-based technique can detect this pattern through reruns alone.

Despite its lower detection rate on unseeded randomness, the variant sweep offers an important practical advantage: it is fully *reproducible*. Because the same ten seeds are always used, two runs on different machines or at different times produce identical results, making the confirmation step deterministic. Naive reruns, by contrast, are probabilistic. A test with a low but non-zero failure rate may not be detected at all in a given ten-run window, leading to inconsistent results across environments.

RQ3: Repair Rate of Rule-Based Transformations

Table 2 and Figure 10 report how many detected flaky tests were successfully repaired by PyIDFix’s Deterministic Patch Generator. A test counts as repaired if the Determinism Validator confirms that the patched version passes consistently across all ten variant seeds.

Table 2: Repair rate by pattern among detected tests.

Pattern	Labeled	Detected	Repaired	Rate
Unordered iteration	7	7	7	100%
Unseeded randomness	4	3	3	100%
Float equality	3	3	3	100%
Overall	14	13	13	100%

Every detected test was repaired successfully, yielding a 100% repair rate across all three pattern categories. The Deterministic Patch Generator applies one of three rule-based transformations depending on the detected pattern: wrapping iteration targets in `sorted()`, inserting `random.seed(42)` at the start of the test function, or replacing exact float comparisons with `pytest.approx()`.

We attribute the perfect repair rate to two factors. First, each of the three patterns has a syntactically well-defined fix that can be expressed as a local, single-site transformation: no cross-function refactoring or change to production code is required. Second, the static analyzer provides the exact line number of the offending expression, so the patch generator can apply the transformation with surgical precision rather than rewriting the entire function. These properties make the patterns well-suited to rule-based repair and suggest that the 100% rate would generalize to other test files exhibiting the same patterns, provided the tests are self-contained and the offending usage is directly visible in the test source.

It is worth noting that the 100% rate may be optimistic with respect to real-world usage. Our benchmark programs were constructed with simple, isolated flakiness patterns, each of which maps cleanly to one transformation. In practice, a test may combine multiple patterns, call into complex library code, or use dynamic constructs such as `eval()` that defeat AST-based analysis. Extending the patch generator to handle compound or indirect patterns remains future work. The one test that was neither detected nor repaired (`test_random_values`) is excluded from the repair-rate denominator, as no patch was generated for it.

RQ4: Patch Correctness and Test Stability

Figure 11 evaluates whether the generated patches (a) eliminate flakiness and (b) leave correctly written tests unaffected.

The left panel of Figure 11 shows a large improvement across all three patterns. Before patching, *unordered iteration* tests pass only 14% of variant seeds on average. This low baseline is consistent with the expectation that Python’s hash randomization produces a different iteration order for most seeds, causing the test’s hard-coded expected sequence to match only rarely. *Unseeded randomness* tests pass only 13% on average, reflecting that test assertions encode specific outputs (i.e. a particular shuffle of a list or a particular random choice) that an unseeded `random` call produces only with low probability. *Float equality* tests show a higher pre-patch average of 67%: two of the three tests always pass on the evaluation machine because the floating-point result happens to be consistent at runtime on this platform, while one always fails because its computed value $((0.1 + 0.2 + 0.3)/3)$ does not actually equal the asserted literal (0.2) in IEEE 754 arithmetic. Their average is therefore $(100 + 100 + 0)/3 \approx 67\%$. After patching, all three patterns reach a 100% average pass rate across the ten seeds, confirming that the transformations completely eliminate the environmental sensitivity for each pattern.

The right panel confirms that all ten deterministic reference tests (`test_deterministic.py` files across all six programs) continued to pass consistently across all ten seeds after patches were applied to the corresponding flaky test files. This is an important negative result: it shows that the tool does not over-patch. We speculate that this stability stems from the narrow scope of each transformation. Adding `sorted()`, inserting a single `random.seed()` call, or swapping `==` for `pytest.approx()` affects only the local assertion and introduces no shared mutable state that could interfere with other tests in the suite.

5 Related Work

Our tool addresses implementation-dependent flaky test detection and repair in Python, which differs fundamentally from the dominant focus on order-dependent (OD) flaky tests in existing research. We categorize related work into three main approaches: order-dependent test detection/repair, implementation-dependent test detection, and hybrid/machine learning approaches.

Order-Dependent Flaky Test Detection and Repair:

iPFlakies [6] is the most closely related work to our project, as it targets Python flaky tests. However, iPFlakies focuses exclusively on order-dependent tests, which pass or fail depending on the execution order of other tests. The framework detects OD tests by running test suites in randomized orders and automatically generates patches by finding “helper” tests that clean polluted shared state. Similarly, iDFlakies [2] and iFixFlakies [8] provide detection and repair for order-dependent tests in Java projects, with iFixFlakies searching for helper tests to reset state and generating patches for 58 out of 110 OD tests. ODRepair [9] extends this approach by using automatic test generation (Randoop) to create cleaners when helper tests don’t exist in the test suite.

The prevalence of flaky tests in both research and industry motivates the need for automated detection and repair tools. Luo et al. [1] conducted one of the first comprehensive empirical studies on flaky tests, analyzing 201 commits from 51 open-source projects and establishing a widely-used taxonomy of root causes. Their study found that flaky tests are pervasive, with major tech companies reporting that flaky tests significantly impact developer productivity and CI/CD reliability.

Our tool differs fundamentally: Rather than detecting flakiness caused by test execution order, we target implementation-dependent flakiness, which fail due to nondeterministic behavior within individual tests themselves (i.e. unordered dictionary/set iteration, unseeded randomness, floating-point equality). These flaky tests fail regardless of execution order and require different detection and repair strategies.

Implementation-Dependent Flaky Test Detection:

NonDex [3, 12] is the only existing tool specifically designed to detect implementation-dependent flaky tests, but it targets Java programs. NonDex detects tests that make incorrect assumptions about underdetermined Java API specifications (e.g., assuming HashSet iteration order is deterministic) by exploring different allowed API behaviors through bytecode instrumentation. The tool has been successfully applied to detect hundreds of ID flaky tests in open-source Java projects by systematically exploring non-deterministic API behaviors allowed by the Java specification.

DexFix [4] extends NonDex by automatically generating domain-specific patches for Java, such as replacing HashMap with LinkedHashMap or adding sorting operations. DexFix represents the state-of-the-art in automated repair for implementation-dependent flaky tests in Java, successfully fixing tests that fail due to wrong assumptions about underdetermined specifications.

Our tool differs in three key ways: First, we target Python rather than Java, which has different sources of nondeterminism (PYTHONHASHSEED, different standard library behaviors). Second, we use static pattern analysis combined with variant-based execution (different hash seeds) rather than bytecode instrumentation. Third, we address a broader range of implementation-dependent patterns including unseeded randomness, floating-point equality, and collection iteration.

Machine Learning and Empirical Approaches:

FlakeFlagger [10] uses machine learning (ML) to predict flaky tests without rerunning, achieving 74% recall and 60% precision by analyzing behavioral features collected during test execution.

CANNIER [11] combines ML predictions with selective rerunning to reduce time cost by 88% while maintaining high detection accuracy.

An empirical study by Gruber et al. [5] specifically examined flaky tests in Python, identifying 7,571 flaky tests across 1,006 Python projects. Their study found that while 59% were order-dependent, a significant portion exhibited other types of flakiness including randomness and infrastructure issues. This empirical evidence demonstrates that implementation-dependent flakiness is a substantial problem in Python projects, yet existing tools like iPFlakies focus primarily on order-dependent tests, leaving a critical gap in automated detection and repair for implementation-dependent flaky tests in Python.

Our tool differs by: (1) Using rule-based static analysis rather than ML for detection, providing deterministic and interpretable results without requiring training data; (2) Confirming flakiness through targeted variant execution rather than repeated reruns, which is more efficient for implementation-dependent patterns; (3) Providing automatic repair through deterministic transformations rather than just detection.

6 Threats to Validity

6.1 Construct Validity

Hash seed only flakiness confirmation

The Execution Variant Generator confirms flakiness by varying PYTHONHASHSEED across ten fixed values. This is the correct lever for unordered iteration flakiness but is ineffective for unseeded randomness (which depends on Python’s random module state, independent of PYTHONHASHSEED) and float equality (which is deterministic on a single platform). Flakiness confirmation rates for these two pattern families are therefore underestimated by the variant sweep, as discussed in RQ2.

6.2 Internal Validity

Benchmark construction bias

The evaluation dataset was generated using a large language model (LLM) and manually reviewed for correctness and pattern coverage. Because the benchmark was designed to contain exactly the three flakiness pattern families PyIDFix targets, detection rates may be inflated relative to real-world test suites. Additionally, LLM-generated code tends to produce canonical, well-structured instances of each pattern, which are likely easier to detect and patch than the messier, more idiomatic instances found in real developer-written test suites. The Stage 1 detection rate therefore reflects performance on a best-case dataset, and generalization to real-world repositories remains unvalidated and is left for future work.

6.3 External Validity

Python version and runtime specificity

All experiments were conducted on a single platform using Python 3.12. Since hash randomization behavior is version-specific, results may differ on older CPython versions where dictionary ordering guarantees and hash semantics differ from those in Python

3.7 and later. Furthermore, alternative Python runtimes may manifest nondeterminism differently or introduce additional sources of flakiness beyond the three patterns PyIDFix currently targets, and the tool’s detection and repair guarantees do not extend to these environments.

Synthetic benchmarks only

All evaluation programs are self-contained synthetic modules created for this study. The tool has not been evaluated on real-world Python repositories. Whether the detection and repair rates observed here generalize to codebases with complex inter-module dependencies, third-party library usage, or the full diversity of ID flakiness patterns found in open-source projects remains an open question. A real-world evaluation would also enable characterizing how PyIDFix scales to large test suites in terms of analysis time and resource usage, and would support a meaningful comparison of ID versus OD flakiness in prevalence and impact, both of which remain important directions for future work.

7 Conclusion

We present PyIDFix, a static analysis tool for the automatic detection and repair of implementation-dependent (ID) flaky tests in Python. While existing tools such as NonDex and DexFix address ID flakiness in Java through bytecode instrumentation, and iPFlakies targets order-dependent Python tests through test reordering, no prior work provides specialized tooling for ID flakiness within Python’s dynamic runtime. PyIDFix fills this gap by combining AST-based source analysis with variant-based execution to identify three prevalent ID patterns (unordered collection iteration, unseeded randomness, and exact float equality) and applying targeted, rule-based patches to eliminate non-determinism at its source without altering test semantics.

The tool operates as a four-stage pipeline: a Static Pattern Analyzer locates offending code, an Execution Variant Generator confirms flakiness by sweeping PYTHONHASHSEED values, a Deterministic Patch Generator rewrites the flagged code, and a Determinism Validator verifies the fix. Evaluated across 6 benchmark programs and 14 labeled flaky tests, PyIDFix achieved a 93% detection rate and a 100% repair rate among detected tests, with analysis completing in under 1.5 ms per file. These results demonstrate that static analysis can surface flakiness that runtime rerun methods are fundamentally incapable of detecting, such as floating-point equality issues that appear deterministic on a single machine but fragile across platforms.

These results are encouraging, though they also surface limitations that point toward several directions for future work. First, implementing lightweight interprocedural analysis will address the current limitation where non-determinism passing through external helper functions goes undetected, which accounted for the single missed detection in our evaluation. Second, extending the pattern catalog to support additional ID patterns including filesystem ordering, environment variable dependencies, and locale-sensitive string comparisons will improve both recall and generality. Third,

conducting a larger-scale evaluation on real-world open-source test suites, including projects constrained from upgrading Python, alongside a comparison with order-dependent flakiness in terms of prevalence and repair complexity, will further validate practical utility. We hope PyIDFix serves as a foundation for future research into automated test maintenance and the broader challenge of making software test suites reliably deterministic.

References

- [1] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, “An Empirical Analysis of Flaky Tests,” in *FSE*, 2014. <https://mir.cs.illinois.edu/lamyaa/publications/fse14.pdf>
- [2] W. Lam, R. Oei, A. Shi, D. Marinov, and T. Xie, “iDFlakies: A Framework for Detecting and Partially Classifying Flaky Tests,” in *ICST*, 2019. <https://ieeexplore.ieee.org/document/8730188/>
- [3] A. Gyori, B. Lambeth, A. Shi, O. Legunsen, and D. Marinov, “NonDex: A Tool for Detecting and Debugging Wrong Assumptions on Java API Specifications,” in *FSE*, 2016. <https://github.com/TestingResearchIllinois/NonDex>
- [4] P. Zhang, Y. Jiang, A. Wei, V. Stodden, D. Marinov, and A. Shi, “Domain-Specific Fixes for Flaky Tests with Wrong Assumptions on Underdetermined Specifications,” in *ICSE*, 2021. <https://ieeexplore.ieee.org/document/9402009/>
- [5] M. Gruber, S. Lukasczyk, F. Krois, and G. Fraser, “An Empirical Study of Flaky Tests in Python,” in *ICST*, 2021. <https://arxiv.org/pdf/2101.09077>
- [6] R. Wang, Y. Chen, and W. Lam, “iPFlakies: A Framework for Detecting and Fixing Python Order-Dependent Flaky Tests,” in *ACM SIGSOFT Software Engineering Notes*, 2022. <https://yangc9.github.io/files/WangETAL22iPFlakies.pdf>
- [7] Kaiyao Ke. “NIODebugger: A Novel Approach to Repair Non-Idempotent-Outcome Tests with LLM-Based Agent,” in *IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025. <https://doi.org/10.1109/ICSE55347.2025.00226>
- [8] A. Shi, W. Lam, R. Oei, T. Xie, and D. Marinov, “iFixFlakies: A Framework for Automatically Fixing Order-Dependent Flaky Tests,” in *ESEC/FSE*, 2019. <https://dl.acm.org/doi/10.1145/3338906.3338925>
- [9] C. Li, C. Zhu, W. Wang, and A. Shi, “Repairing Order-Dependent Flaky Tests via Test Generation,” in *ICSE*, 2022. <https://wenxiwang.github.io/papers/ICSE2022.pdf>
- [10] A. Alshammari, C. Morris, M. Hilton, and J. Bell, “FlakeFlagger: Predicting Flakiness Without Rerunning Tests,” in *ICSE*, 2021. <https://www.jonbell.net/preprint/icse21-flakeflagger.pdf>
- [11] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, “Empirically Evaluating Flaky Test Detection Techniques Combining Test Case Rerunning and Machine Learning Models,” *Empirical Software Engineering*, 2023. <https://link.springer.com/article/10.1007/s10664-023-10307-w>
- [12] A. Shi, A. Gyori, O. Legunsen, and D. Marinov, “Detecting Assumptions on Deterministic Implementations of Non-deterministic Specifications,” in *ICST*, 2016. <https://mir.cs.illinois.edu/awshi2/publications/FSEDEMO2016.pdf>
- [13] J. Sivaloganathan, A. Jamshidi, A. Miranskyy, and L. Zhang, “Automating Detection and Root-Cause Analysis of Flaky Tests in Quantum Software,” *arXiv preprint arXiv:2603.09029*, 2026. <https://arxiv.org/pdf/2603.09029>

RQ1 — Static Analysis vs. Naive Reruns

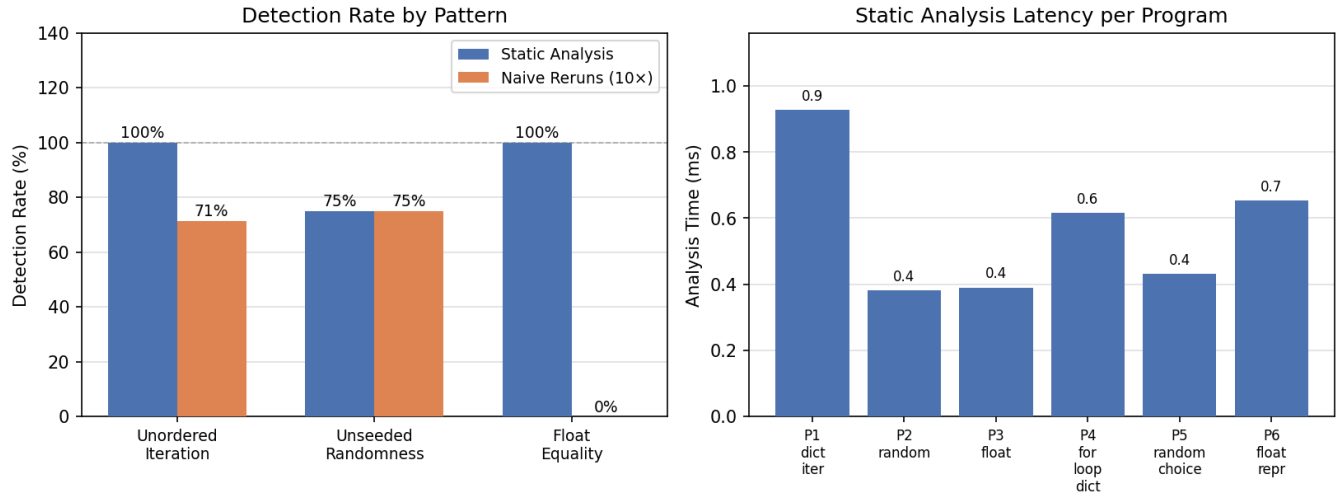


Figure 8: RQ1. *Left*: detection rate (%) by flakiness pattern for the Static Pattern Analyzer (blue) vs. a naive rerun baseline (orange, 10 runs per test with randomized PYTHONHASHSEED). *Right*: average static analysis latency per program (ms).

RQ2 — Flaky-Test Detection Rate: Variant Sweep vs. Naive Reruns

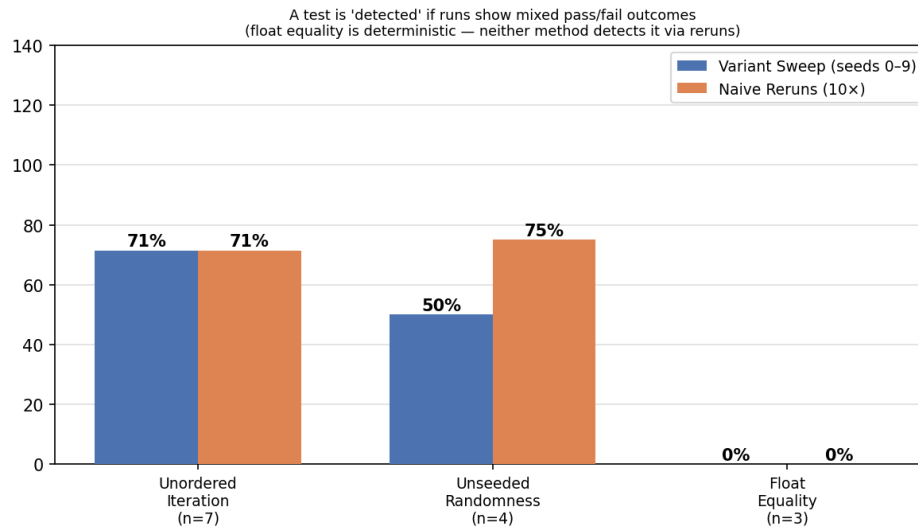


Figure 9: RQ2. Flaky-test detection rate (%) by pattern for the Execution Variant Generator (structured seed sweep 0-9, blue) vs. naive reruns (random PYTHONHASHSEED, orange). *n* denotes the number of labeled tests per pattern. Detection requires mixed pass/fail outcomes across runs.

RQ3 — Automated Repair Rate

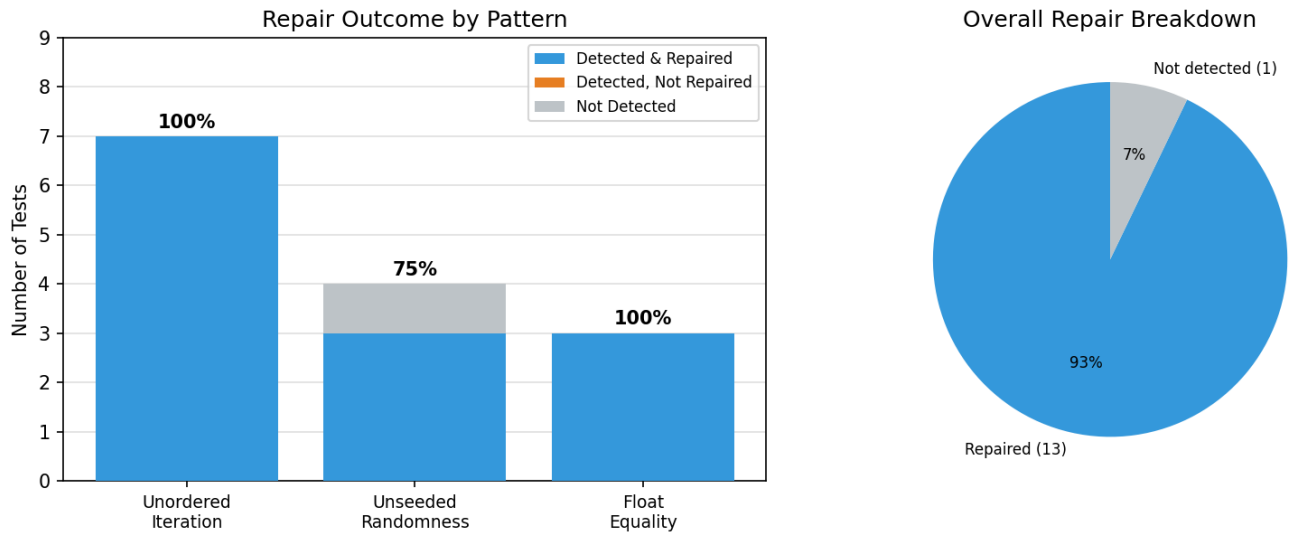


Figure 10: RQ3. *Left*: stacked bars showing repair outcome per pattern (blue = detected and repaired, grey = not detected). Percentages above each bar indicate the repair rate among detected tests. *Right*: overall breakdown across all 14 labeled tests.

RQ4 — Patch Correctness: Pass Rate Before vs After

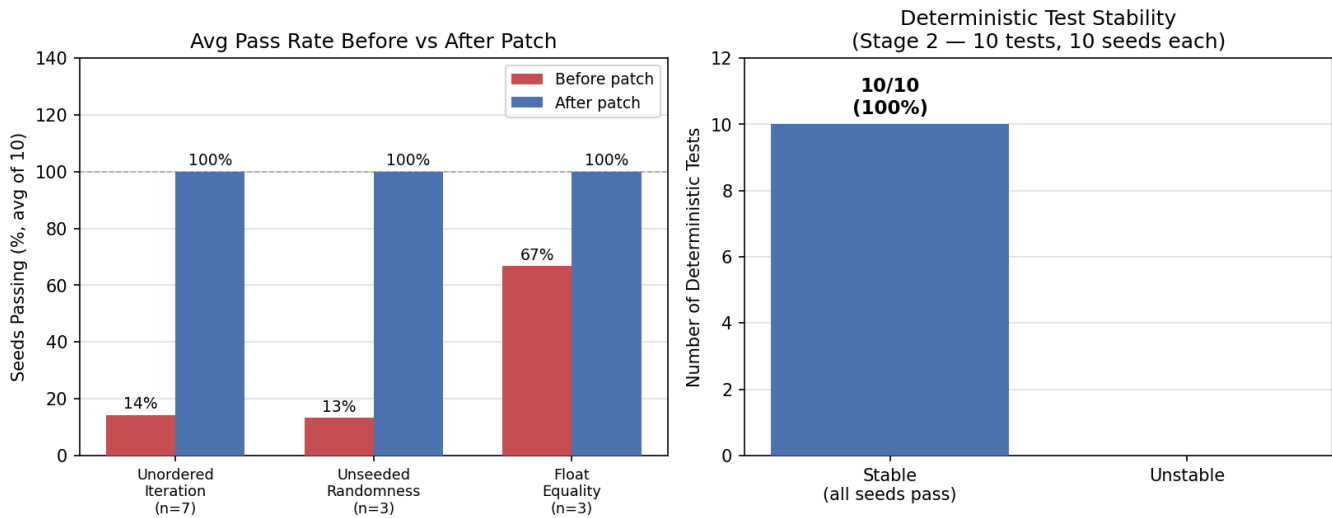


Figure 11: RQ4. *Left*: average pass rate across ten variant seeds before (red) and after (blue) patching, grouped by pattern. *n* = number of tests contributing to each average; the dashed line marks the 100% target. *Right*: deterministic reference tests (Stage 2) that remained stable across all seeds after patching.